

NPRG086: Functions, trees, processes

Course introduction

Tomáš Petříček, 204 (2nd floor)

✉ petricek@d3s.mff.cuni.cz

🦋 [@tomaspetricek.net](https://tomaspetricek.net)

🌐 <https://tomaspetricek.net>



Who are we

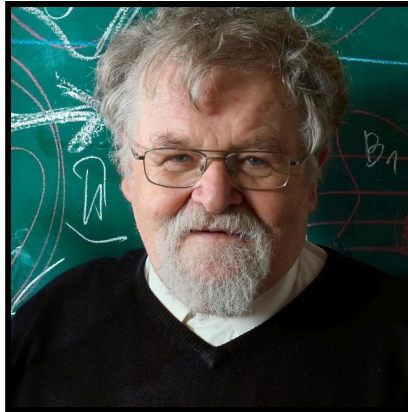
David Corfield

philosophy of
mathematics



Jaroslav Nesetril

discrete
mathematics



Tomas Petricek

programming
languages



Computer science concepts

effect

process

function

flow

tree

object

graph

type

monad

event

address

Philosophy

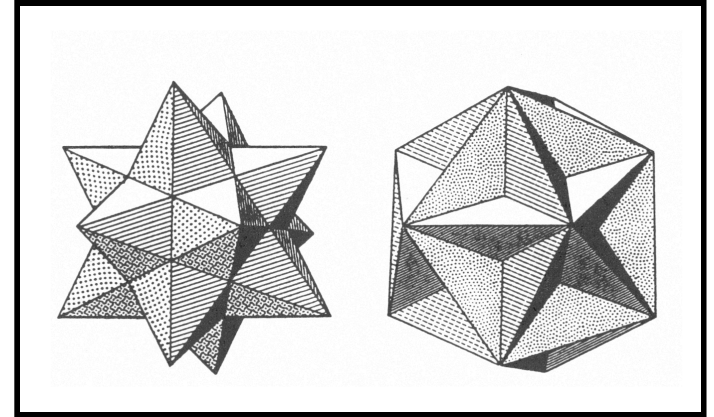
Computer science concepts

- 🌲 Where do they come from?
- 🕒 How do they change over time?
- 💬 Do we agree what they mean?
- 🧪 How they cross disciplines?

Philosophy of mathematics and physics

Mathematics

- Polyhedra ($E = V + F - 2$)
- Functions (non-differentiable)
- Groups (monster group)



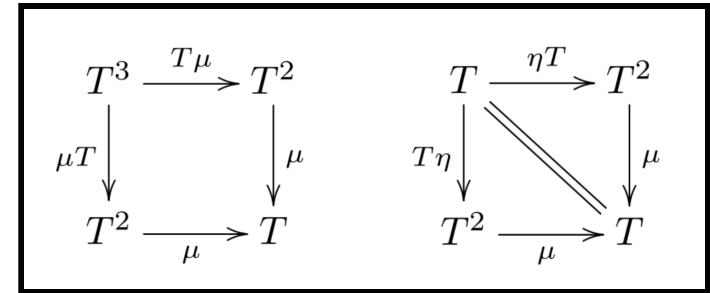
Physics

- Phlogiston (fire-like element)
- Electrons (can spray them)
- Temperature (and measurement)

Why three people

Monad

- Originates in category theory
- Used in formal semantics
- Implemented in Haskell and C#



Tree

- Minimum spanning tree
- B-tree in databases and file systems
- Decision trees in machine learning

Course

Practical info

Course timeline

- Week 1 (1 Oct) - Introduction
- ~~Week 2 (8 Oct) - Cancelled due to travel~~
- Week 3 (15 Oct) - Functions in mathematics (DC)
- Week 4 (22 Oct) - Functions in early days of programming (TP)
- Week 5 (29 Oct) - Functions and constructivism (DC)
- Week 6 (5 Nov) - Functional programming (TP)
- Week 7 (12 Nov) - Monads and their origins (DC)
- Week 8 (19 Nov) - Monads in functional programming (TP)
- Week 9 (26 Nov) - Monads, continuations, constructivism (DC)
- Week 10 (3 Dec) - Trees as a programming construct (TP)
- Week 11 (10 Dec) - Trees and their origins (JN)
- Week 12 (17 Dec) - Trees in discrete mathematics (JN)
- Week 13 (7 Jan) - Trees in discrete mathematics (JN)

How to get credits

The exam can be completed either as an oral exam covering topics discussed during the given semester, or as an individual project on a topic agreed with the instructors.

Individual project

- Look at a concept or entity of your own choice!
- Look at a perspective that we've missed!
- Talk to us to discuss practical arrangements
- Write about it or present your findings

Background

History of computing

Histories of programming

First hand accounts

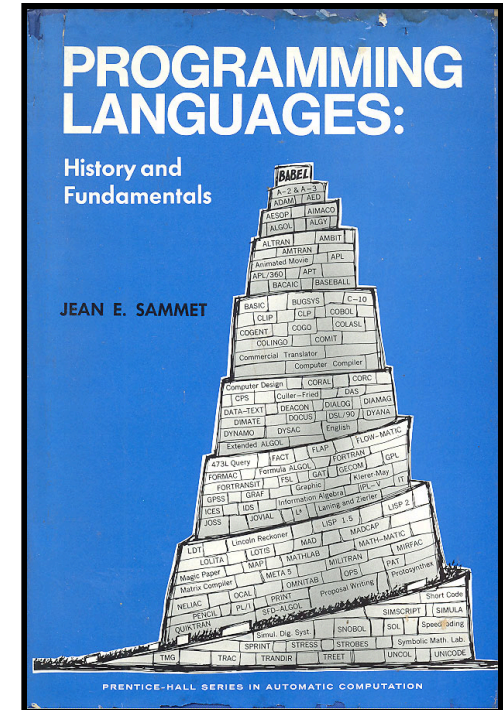
- Framing and own history
- ACM HOPL, talks and blogs

History of science

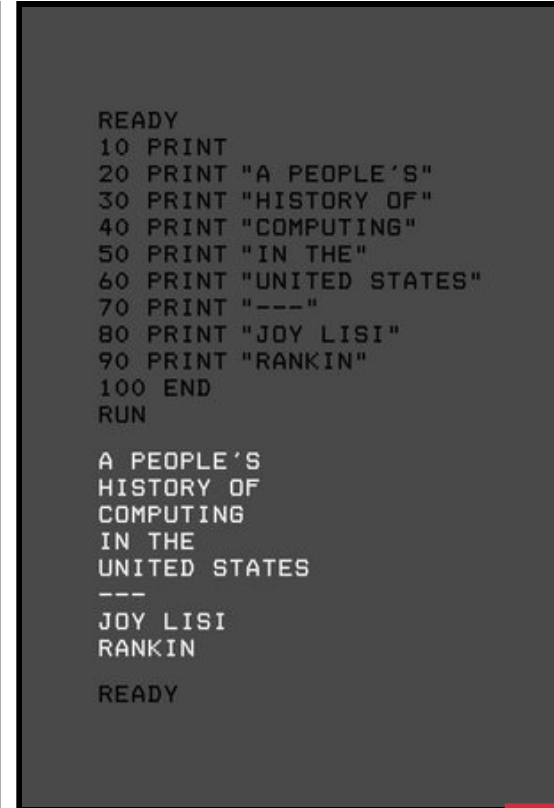
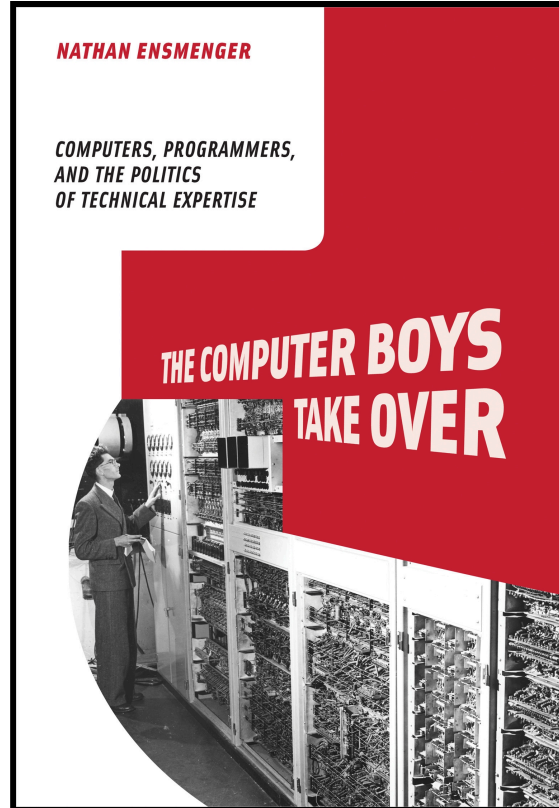
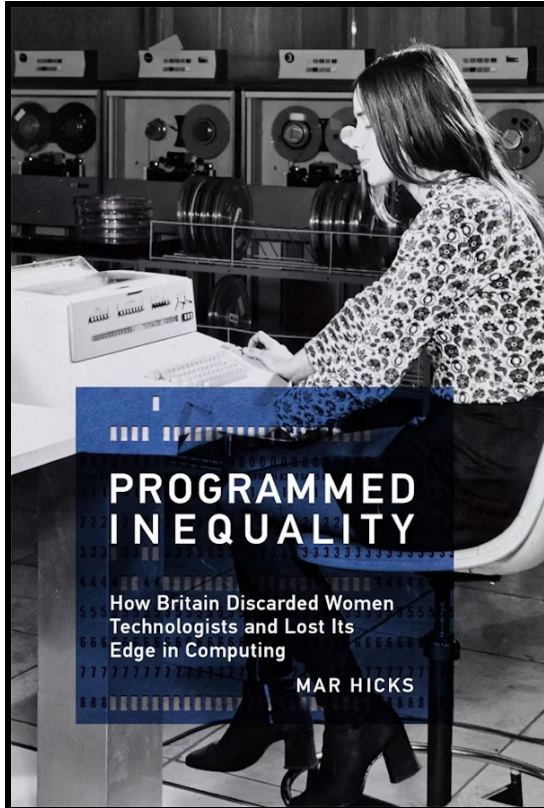
- History from technical perspective
- Each claim needs hard evidence!

Technology and society

- Broader socio-technical issues
- Archives, letters, oral interviews



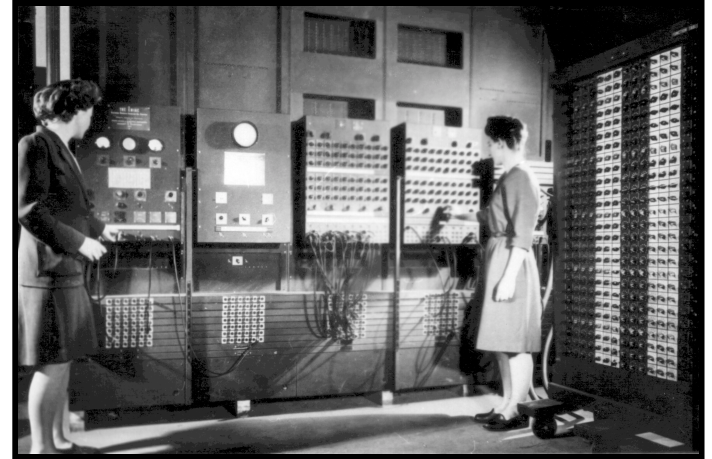
Science, technology, society (STS)



Programming the ENIAC

ENIAC (1945)

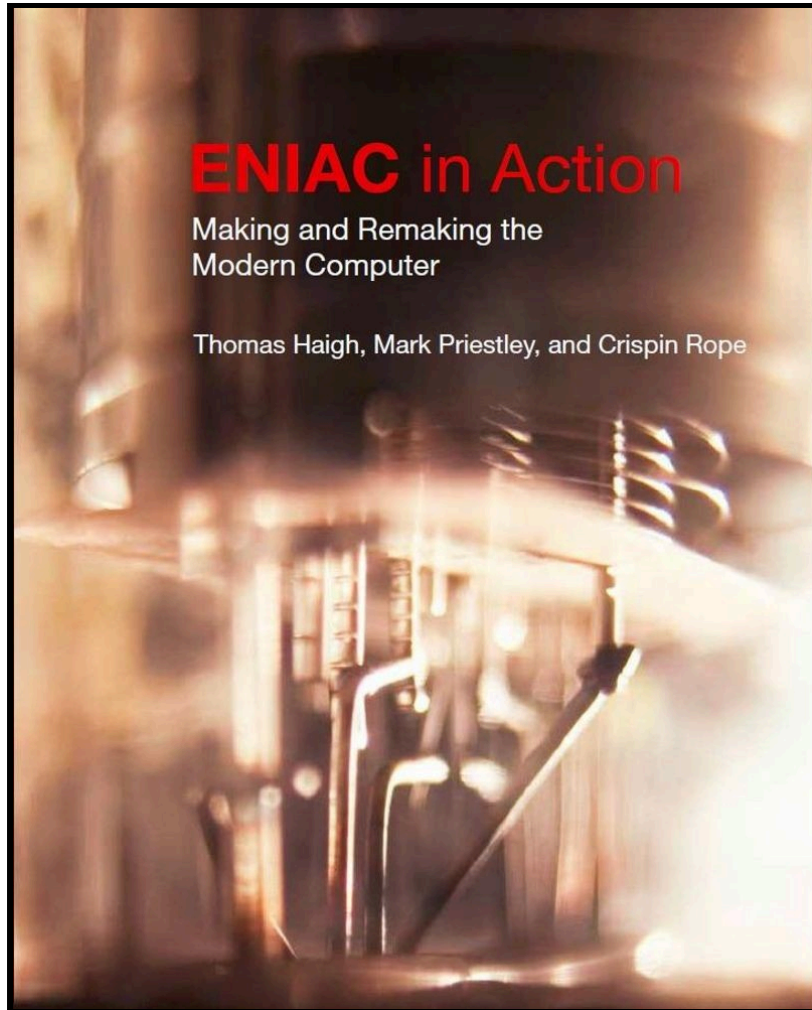
- First "programmable, electronic, general-purpose digital" computer
- Programmed by plugging cables



Questioning the "first"

- Program not stored in memory
- Not that different from Harvard Mark I?

How exactly did the programming work?



History of the ENIAC

Document design,
construction, testing,
and use by the military

Reconfiguration of
ENIAC in 1947-48
and the atomic
fission simulation

Table 7.2 (continued)

Control instructions

69	C.T.	Conditional transfer. If accumulator 15 is negative then continue with the next instruction. Otherwise, set current instruction address from the future control address to jump.
94, 85, 96	i, di, cdi	Control the master programmer. ^a
75abcd	N3D8	Add 4 inline digits to the function-table address.
90, 99	D	Delay: go to next instruction.
00, 35	H	Halt the machine.

a. By 1951, these instructions had been added to allow loops to be set up on the master programmer (which was not fully utilized by the earlier converter codes) and controlled using the pre-conversion mechanisms.

Conversion

Inspired by later
EDVAC report

"the storage of
programming data
by means of the
function table"

51 Order Code
developed in 1947

Histories

Functional programming

Popular history of functional programming

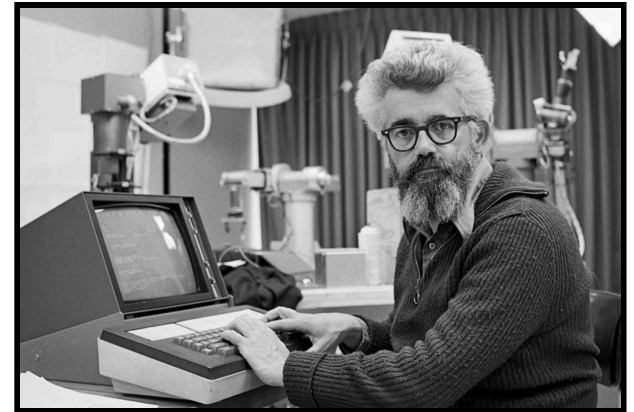
Alonzo Church (1936)

Lambda calculus model
based on pure functions



John McCarthy (1960)

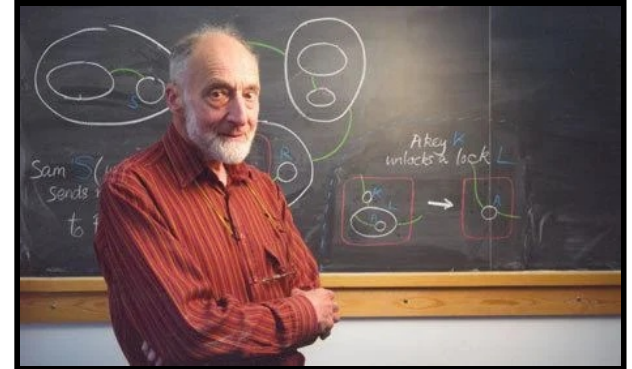
Creates the LISP language
based on the lambda calculus



Popular history of functional programming

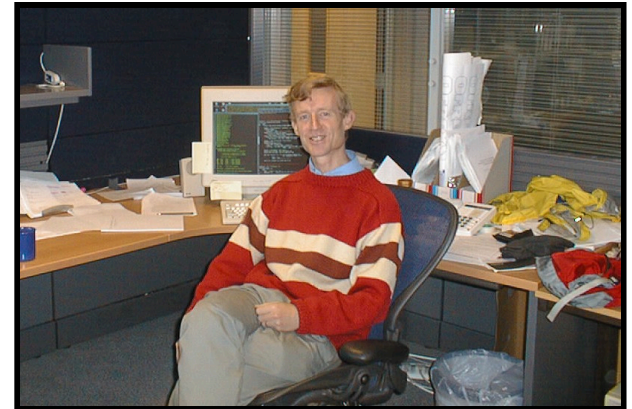
Robin Milner (1973)

ML language adds static types with automatic type inference



Simon Peyton Jones (1990)

Created Haskell, a statically-typed and lazy functional language



What is wrong?

Just about everything...

Narrow "great men" reading

None of these were created by a single person

Retrospective reading

Lambda calculus not a programming language!

Many claims factually wrong

LISP had AI origins, not lambda calculus

Invents a story of progress

Towards static types and laziness

Histories

Looking more carefully

Lambda calculus and programming

Origins of lambda calculus (1930s)

- Foundations of mathematics (Hilbert)
- Russell's paradox $R = \{x \mid x \notin x\}$
- Avoid paradox without types
- Based on functions not sets

Lambda calculus is inconsistent

- Kleene-Rosser paradox
- Let $R = \lambda x. \neg(x x)$ then $R R = \neg(R R)$
- Does not work! Unless you add types ala Russell



LISP origins

FOTRAN for numerical computing, COBOL for business records

Manipulation of symbolic expressions in the context of good old fashioned AI

Initial project was an addition to for FOTRAN

LISP I
PROGRAMMER'S MANUAL
March 1, 1960

Artificial Intelligence Group

J. McCarthy
R. Brayton
D. Edwards
P. Fox
L. Hodes
D. Luckham
K. Maling
D. Park
S. Russell

LISP origins

Demystifying the history of LISP

- ☰ Only thing from lambda calculus is **LAMBDA**
- 📍 Had assignments and **goto** before conditionals
- “ Used first-order M-expressions at first
- √x Had dynamic scoping until Scheme (1975)

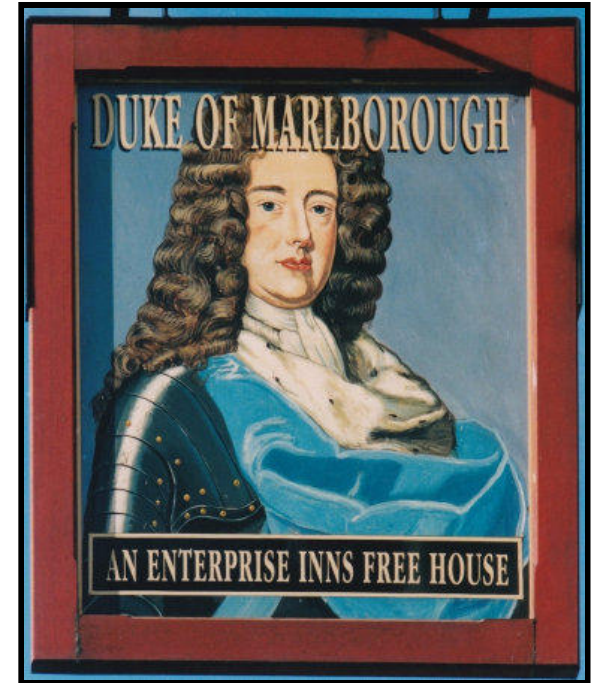
Rediscovering lambda calculus

Mervyn Pragnell's group

- Unofficial logic reading group
- Got to know about the λ -calculus
- Penrose $\rightarrow$ Strachey $\rightarrow$ Landin

Landin and lambda calculus

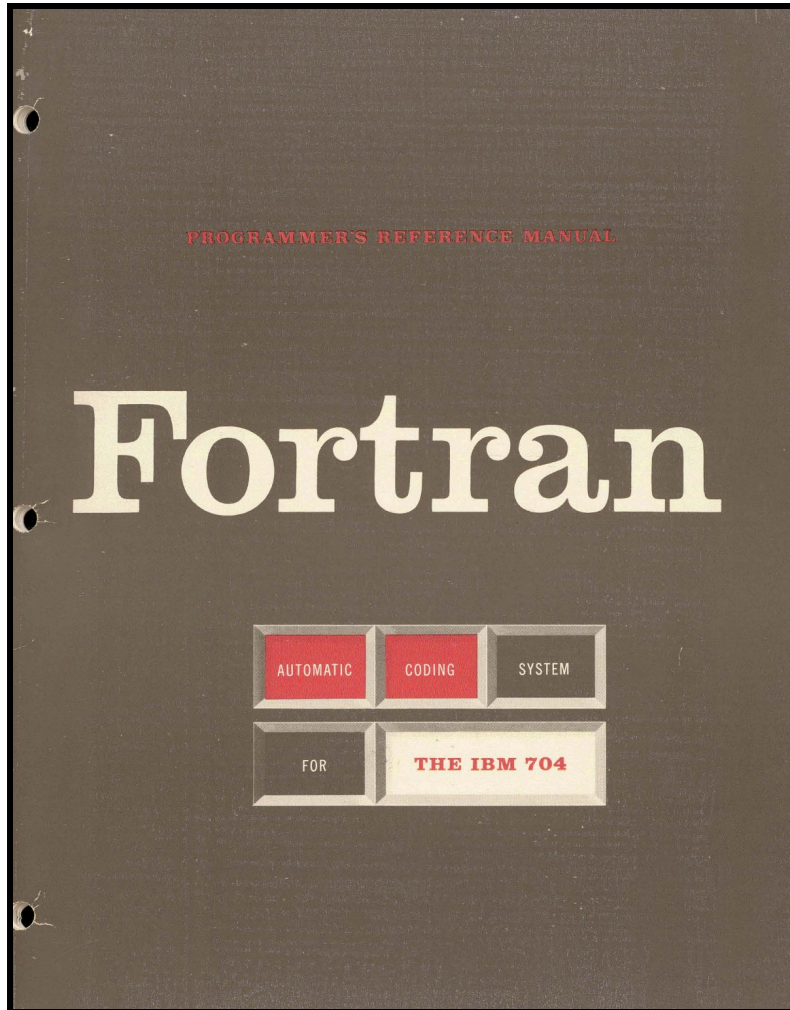
- Formal model of the Algol language
- Mostly forgotten languages...
- Work on abstract data types



Origins of types

FORTRAN for IBM 704

[A function] may be desired to state the argument either in **fixed or floating point**; similarly the function itself may be in either of these **modes**. Thus a function of a single argument has 4 possible mode configurations.



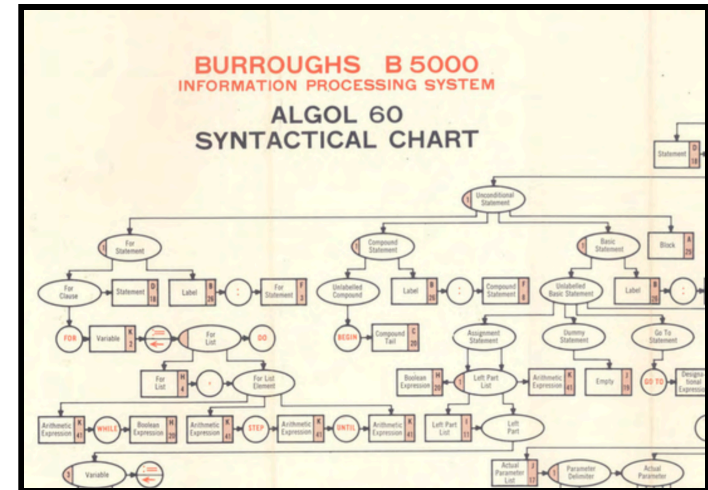
Types appear in Algol 58

May 58 meeting

- European and US group meeting
- None uses *type* in prior reports

Preliminary report

No reference to logic or λ -calculus



Integers are of type integer. All other numbers are of type real. (...) The various "types" (integer, real, Boolean) basically denote properties of values.

What are types?

TYPES ARE NOT SETS *

James H. Morris, Jr.
Xerox Corporation
Palo Alto Research Center (PARC)
Palo Alto, California 94304

Type checking is a way to prevent such things from happening. All values used to represent the abstract objects are considered to be of a certain type. The rules are:

- (1) Only values of that type can be submitted for processing (authentication).
- (2) Only the procedures given can be applied to objects of that type (secrecy).

Sets of possible values

Cartesian products
to model records

Types are not sets (1974)

Inspired by Landin's work
but does not cite typed
lambda calculus

Logic of computable functions

Dana Scott (1969)

- A type-theoretical alternative to ISWIM, CUCH, OWHY (unpublished manuscript)
- Logic for reasoning about computation



Ensures meaning using types

The first confusion we should avoid is that between logical types and what we might call data types. The former are what we invoke to study the latter.

Types

Towards types in ML

-  Directly inspired by the LCF logic
-  Authentication and secrecy for proof objects
-  Unaware of a lot of work in logic (Hindely-Milner)
-  Later additions add tuples and data types

Conclusions

Functions, trees, processes

Conclusions

-  We need rigorous technical history too!
Complement first-hand accounts and social history
-  Programming entities are interesting!
They change meaning and cross boundaries
-  Starting with functions, monads and trees
Help us look deeper and broader!

